1. What is an Algorithm?

An algorithm is a sequence of **unambiguous instructions** for solving a problem, that is, for obtaining a required output for any legitimate input in a **finite amount of time**.
An algorithm provides a step-by-step procedure that transforms the given input into the desired output and terminates after a finite number of steps.

Characteristics of an Algorithm:

- **Input:** An algorithm should accept one or more inputs.
- **Output:** It should produce at least one output.
- **Definiteness:** Each instruction must be clear and unambiguous.
- **Finiteness:** The algorithm must terminate after a finite number of steps.
- **Effectiveness:** Each instruction should be basic and executable.

Diagram: Notion of Algorithm
Reference Material: Anany Levitin – *Introduction to the Design and Analysis of Algorithms*
Page No: 3

2. Write Euclid's algorithm for GCD calculation.

Euclid's algorithm is used to find the **greatest common divisor (GCD)** of two non-negative integers *m* and *n*, where both are not zero.
It is based on the principle:

$$\gcd(m, n) = \gcd(n, m \bmod n)$$

Algorithm: Euclid(m, n)

Input: Two non-negative integers m and n, not both zero
Output: Greatest common divisor of m and n
while n ≠ 0 do
r ← m mod n
m ← n
n ← r
return m

3. What is Algorithm Design Technique?

Algorithm design techniques are **general problem-solving strategies** used to design algorithms for computational problems. These techniques classify algorithms based on their **design approach**, rather than on the type of problem being solved.

Common algorithm design techniques include:

- Brute Force
- Divide and Conquer
- Decrease and Conquer
- Transform and Conquer
- Greedy Technique
- Dynamic Programming
- Iterative Improvement

4. Differentiate Time Efficiency and Space Efficiency.

Basis	Time Efficiency	Space Efficiency
Meaning	**Measures how much time an algorithm takes to run.**	**Measures how much memory an algorithm uses.**
Focus	**Speed of execution.**	**Memory usage.**
Measured By	**Number of basic operations performed.**	**Amount of memory space required.**
Depends On	**Size of the input.**	**Input size and extra memory used by the algorithm.**
Concerned With	**How fast an algorithm works.**	**How much memory an algorithm consumes.**

5. How will you measure input size of algorithms?

The input size of an algorithm is measured based on the **amount of data provided** to the algorithm. The method of measurement depends on the nature of the input:

- For an **array or list**: input size is the number of elements (n).
- For a **matrix**: input size is represented by the number of rows and columns.
- For a **string**: input size is the number of characters.
- For an **integer**: input size is measured by the number of bits required to represent the integer.

6. Define best, worst and average case efficiency.

Best Case Efficiency:
Best case efficiency is the **minimum time** required by an algorithm to execute for an input of size *n*. It occurs for the most favorable input.

Worst Case Efficiency:
Worst case efficiency is the **maximum time** required by an algorithm to execute for an input of size *n*. It represents the upper bound on running time.

Average Case Efficiency:
Average case efficiency is the **expected time** taken by an algorithm over all possible inputs of size *n*, assuming a uniform distribution of inputs.

7. Define Big-O, Big-Omega and Big-Theta notations.

Big-Oh (O) Notation:
Big-Oh notation represents the **upper bound** on the running time of an algorithm. It describes the worst-case growth rate.

Big-Omega (Ω) Notation:
Big-Omega notation represents the **lower bound** on the running time of an algorithm. It describes the best-case growth rate.

Big-Theta (Θ) Notation:
Big-Theta notation represents the **tight bound** on the running time of an algorithm. It indicates that the algorithm grows at the same rate in both upper and lower bounds.

8. List the basic efficiency classes.

The basic efficiency classes of algorithms are:

- Constant time – **O(1)**
- Logarithmic time – **O(log n)**
- Linear time – **O(n)**
- Linearithmic time – **O(n log n)**
- Quadratic time – **$O(n^2)$**
- Cubic time – **$O(n^3)$**
- Exponential time – **$O(2^n)$**
- Factorial time – **O(n!)**

9. Define recurrence relation.

A **recurrence relation** is an equation that defines a function in terms of its value on **smaller inputs**.
It is commonly used to represent the time complexity of **recursive algorithms**.

10. Define non-recursive algorithm.

A **non-recursive algorithm** is an algorithm that does **not call itself** directly or indirectly.
It uses iterative constructs such as **loops** to solve the problem.

11. Write an algorithm using recursive function to find the sum of n natural numbers.

Algorithm: Sum(n)

ALGORITHM: Sum(n)

Purpose: To find the sum of first *n* natural numbers using recursion.

1. **Start**
2. **Read** the value of n
3. **If** n = 0
    - Return 0
4. **Else**
    - Return n + Sum(n − 1)
5. **Stop**

12. List the factors which affect the running time of the algorithm.

The factors that affect the running time of an algorithm are:

- Size of input
- Nature of input data
- Algorithm design technique used
- Number of basic operations
- Hardware and system architecture

- Programming language used

.

13. What is meant by substitution method?

The **substitution method** is a technique used to solve recurrence relations.
In this method, a solution is first **guessed**, and then the guess is **proved using mathematical induction**.

14. Discuss in detail about fundamentals of algorithmic problem solving.

14. Discuss in detail the fundamentals of algorithmic problem solving.

The fundamentals of algorithmic problem solving include the following steps:

1. **Understanding the Problem:**
   Clearly identify the problem requirements and constraints.
2. **Ascertaining the Capabilities of the Computational Device:**
   Consider limitations of memory, speed, and input/output.
3. **Choosing between Exact and Approximate Solutions:**
   Decide whether an exact solution or an approximate solution is acceptable.
4. **Deciding on Appropriate Data Structures:**
   Select suitable data structures such as arrays, lists, trees, or graphs.
5. **Algorithm Design Techniques:**
   Choose an appropriate technique such as brute force or divide and conquer.
6. **Methods of Specifying an Algorithm:**
   Use pseudocode, flowcharts, or natural language.
7. **Proving Algorithm Correctness:**
   Ensure the algorithm produces correct results for all valid inputs.
8. **Analyzing the Algorithm:**
   Evaluate time and space efficiency.
9. **Coding the Algorithm:**
   Convert the algorithm into an executable program.

15. Explain the important problem types in detail.

15. Explain the important problem types in detail.

In algorithm design and analysis, problems are commonly classified into several important types based on the nature of computation involved. These problem types help in selecting suitable algorithm design techniques.

---

1. Sorting Problems

Sorting problems deal with **rearranging elements of a list** into a specific order, usually **ascending or descending**.
The input is a sequence of elements, and the output is the same sequence arranged in the desired order.

Sorting is important because:

- It makes **searching faster**
- Many algorithms use sorting as a **preprocessing step**

Examples of sorting problems include arranging numbers, names, or records based on a key. Common sorting algorithms are selection sort, bubble sort, insertion sort, merge sort, and heap sort.

---

2. Searching Problems

Searching problems involve **finding the location of a specific element (key)** within a collection of data.
The output may be the position of the element or a statement indicating whether the element is present or not.

Searching problems are fundamental in computer science and are widely used in:

- Databases
- File systems
- Information retrieval

Examples include linear search and binary search.

---

3. String Processing Problems

String processing problems deal with **operations on strings**, which are sequences of characters. These problems are common in text processing, document editing, and pattern matching.

Typical string processing tasks include:

- Finding a pattern within a text
- Comparing strings
- Manipulating strings

An example is **brute force string matching**, where a pattern is searched within a larger text by comparing characters one by one.

---

4. Graph Problems

Graph problems involve data represented in the form of **vertices (nodes) and edges**.
They are used to model relationships such as networks, paths, and connections.

Graph problems are important in areas such as:

- Computer networks
- Transportation systems
- Social networks

Examples of graph problems include:

- Finding the shortest path
- Spanning tree problems
- Travelling Salesman Problem

---

5. Combinatorial Problems

Combinatorial problems involve **finding combinations, permutations, or subsets** of a given set of elements.
These problems often have a very large number of possible solutions.

Such problems are usually:

- Computationally expensive
- Solved using brute force, backtracking, or dynamic programming

Examples include generating all permutations of a set and the knapsack problem.

---

6. Numerical Problems

Numerical problems deal with **mathematical computations** and numerical data.
These problems require precise calculations and are common in scientific and engineering applications.

Examples of numerical problems include:

- Computing the greatest common divisor (GCD)
- Matrix operations
- Solving equations

.

16. Explain asymptotic notations with examples.

Asymptotic notations are used to describe the growth rate of an algorithm's running time as input size increases.

- **Big-Oh (O) notation** is used to describe the **worst-case performance** of an algorithm.
  Example: Linear search has worst-case complexity O(n).
- **Big-Omega (Ω) notation** is used to describe the **best-case performance** of an algorithm.
  Example: Linear search has best-case complexity Ω(1).
- **Big-Theta (Θ) notation** is used to describe the **average or tight bound** of an algorithm.
  Example: Linear search has average-case complexity Θ(n).

OR

Asymptotic notations are used to describe the **growth rate of an algorithm's running time** as the input size increases, ignoring constant factors and lower-order terms.

Big-Oh (O) – Worst Case Analysis

Big-Oh notation represents the **upper bound** on the running time of an algorithm and is used to analyze the **worst-case performance**.

Example Algorithm: Linear Search (Worst Case)

ALGORITHM LinearSearch(A, n, key)
Input: Array A of n elements, key
Output: Position of key or −1

```
for i ← 0 to n−1 do
   if A[i] = key then
      return i
return −1
```

Explanation (2 sentences):
In the worst case, the key is not present or is located at the last position of the array. Therefore, the algorithm compares all n elements, giving a time complexity of **O(n)**.

Big-Omega (Ω) – Best Case Analysis

Big-Omega notation represents the **lower bound** on the running time of an algorithm and is used to analyze the **best-case performance**.

Example Algorithm: Linear Search (Best Case)

ALGORITHM LinearSearch(A, n, key)
Input: Array A of n elements, key
Output: Position of key

```
if A[0] = key then
   return 0
```

Explanation (2 sentences):
In the best case, the key element is found at the first position of the array. Hence, only one comparison is required and the time complexity is **Ω(1)**.

Big-Theta (Θ) – Average Case / Tight Bound Analysis

Big-Theta notation represents a **tight bound**, indicating that the algorithm's running time grows at the same rate in both upper and lower bounds.

Example Algorithm: Linear Search (Average Case)

ALGORITHM LinearSearch(A, n, key)
Input: Array A of n elements, key
Output: Position of key or −1

for i ← 0 to n−1 do

  if A[i] = key then

    return i

return −1

Explanation (2 sentences):
On average, the key is found after checking about n/2 elements. Therefore, the average-case time complexity of linear search is **Θ(n)**.

17. Explain the method of solving recurrence equations.

Recurrence equations are commonly used to analyze recursive algorithms.
Methods for solving recurrence relations include:

- Substitution method
- Iteration method

Example:
T(n) = T(n − 1) + c

By expanding repeatedly, we get:
T(n) = cn + c
Hence, the time complexity is **O(n)**.

OR

Recurrence equations are used to express the time complexity of recursive algorithms in terms of smaller input sizes. They are commonly solved using methods such as substitution and iteration.

Example Algorithm: Recursive Sum of n Numbers

ALGORITHM Sum(n)
Input: A non-negative integer n
Output: Sum of first n natural numbers

if n = 0 then

  return 0

else

return n + Sum(n − 1)

Recurrence Relation:
$T(n) = T(n - 1) + c$

Explanation (2 sentences):
Each recursive call reduces the problem size by 1 and performs a constant amount of work.
Solving the recurrence gives $T(n) = O(n)$

18. Explain the method of solving non-recursive equations.

For non-recursive algorithms, the running time is analyzed by:

1. Identifying the **basic operation**
2. Counting how many times the operation is executed
3. Expressing the count as a summation
4. Determining the order of growth

Example:
A loop that runs from 1 to n executes n times → Time complexity O(n).

OR

Non-recursive algorithms are analyzed by counting how many times a **basic operation** is executed. This count is expressed using summations to determine the order of growth.

Example Algorithm: Finding Maximum Element in an Array

ALGORITHM FindMax(A, n)
Input: Array A of n elements
Output: Maximum element

max ← A[0]

for i ← 1 to n−1 do

```
    if A[i] > max then
        max ← A[i]
return max
```

Explanation (2 sentences):
The loop executes n−1 times and performs one comparison in each iteration. Hence, the time complexity of the algorithm is **O(n)**.

19. Describe the basic efficiency classes.

Basic efficiency classes describe how algorithm performance grows with input size:

- O(1) – Constant time
- O(log n) – Logarithmic time
- O(n) – Linear time
- O(n log n) – Linearithmic time
- $O(n^2)$ – Quadratic time
- $O(n^3)$ – Cubic time
- $O(2^n)$ – Exponential time
- O(n!) – Factorial time
-

OR

O(1) – Constant time:
The running time of the algorithm remains constant and does not depend on the input size.

O(log n) – Logarithmic time:
The running time increases logarithmically as the input size increases, usually by repeatedly reducing the problem size.

O(n) – Linear time:
The running time increases directly in proportion to the input size.

 O(n log n) – Linearithmic time:
The running time grows proportional to n log n and commonly occurs in efficient sorting algorithms.

 O(n^2) – Quadratic time:
The running time increases proportionally to the square of the input size, often due to nested loops.

 O(n^3) – Cubic time:
The running time increases in proportion to the cube of the input size, typically seen in algorithms with three nested loops.

 O(2^n) – Exponential time:
The running time doubles with each additional input element, making the algorithm impractical for large inputs.

 O(n!) – Factorial time:
The running time grows factorially with input size and is usually associated with exhaustive permutation-based algorithms

.

20. What are the conditions for Travelling Salesman Problem?

The conditions for the Travelling Salesman Problem (TSP) are:

- The salesman must visit **each city exactly once**
- The salesman must return to the **starting city**
- The total travel cost must be **minimum**
- The problem is represented using a **complete weighted graph**

Diagram: Weighted Graph Representation of TSP
Reference Material: Travelling Salesperson Problem – Module Reference PDF
Page No: As given in the TSP module notes

21. Define Knapsack problem.

The Knapsack Problem is defined as follows:
Given *n* items, each with a weight and a value, select a subset of items such that the total weight does not exceed the knapsack capacity and the total value is maximized.

Diagram: Knapsack Representation (Items–Weight–Value)
Reference Material: Knapsack Problem – Module 1 PDF
Page No: As given in the knapsack module notes

22. Explain selection sort and bubble sort.

Both **Selection Sort** and **Bubble Sort** are **brute force sorting algorithms**, as they sort data by repeatedly comparing elements without using any advanced optimization techniques.

Selection Sort

Explanation:
Selection Sort works by repeatedly selecting the **smallest element** from the unsorted portion of the list and placing it at the correct position in the sorted portion. At every step, one element is fixed in its final position.

Algorithm: Selection Sort

ALGORITHM SelectionSort(A, n)
Input: Array A of n elements
Output: Sorted array in ascending order

```
for i ← 0 to n − 2 do
    min ← i
    for j ← i + 1 to n − 1 do
        if A[j] < A[min] then
            min ← j
    swap A[i] and A[min]
```

Example:
For the input array {7, 5, 4, 2}, the algorithm first places 2 at the first position, then 4, followed by 5, and finally 7, resulting in a sorted array.

Bubble Sort

Explanation:
Bubble Sort works by repeatedly comparing **adjacent elements** and swapping them if they are in

the wrong order. After each pass, the largest element "bubbles" to its correct position at the end of the list.

Algorithm: Bubble Sort

ALGORITHM BubbleSort(A, n)
Input: Array A of n elements
Output: Sorted array in ascending order

```
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j] > A[j + 1] then
            swap A[j] and A[j + 1]
```

Example:
For the input array {5, 3, 8, 4}, repeated comparisons and swaps move the largest element to the end in each pass until the array becomes sorted.

23. Describe sequential search and brute force string matching.

Sequential Search (Linear Search)

Sequential search is a **brute force searching technique** in which each element of the list is compared **one by one** with the given key element until the key is found or the list ends. It does not require the data to be sorted and is simple to implement, but it is inefficient for large datasets.

Algorithm: Sequential Search

ALGORITHM LinearSearch(A, n, key)
Input: Array A of n elements and search key
Output: Position of key or −1 if not found

```
for i ← 0 to n − 1 do
    if A[i] = key then
        return i
return −1
```

In the best case, the element is found in the first position, while in the worst case, all elements must be checked.

Brute Force String Matching

Brute force string matching is a **simple pattern matching technique** where the pattern is aligned with the text at every possible position and characters are compared **one by one**. If a mismatch occurs, the pattern is shifted by one position and the comparison process is repeated.

Algorithm: Brute Force String Matching

ALGORITHM BruteForceStringMatch(T, P)
Input: Text T of length n and Pattern P of length m
Output: Starting position of pattern or −1

```
for i ← 0 to n − m do
    j ← 0
    while j < m and T[i + j] = P[j] do
        j ← j + 1
    if j = m then
        return i
return −1
```

This method is easy to understand and implement but becomes inefficient for large texts due to repeated comparisons.

24. Describe exhaustive search in detail.

Exhaustive search is a **brute force algorithm design technique** in which **all possible candidate solutions** of a problem are generated and examined one by one. Each candidate solution is tested to check whether it satisfies the given constraints, and the solution that best meets the objective of the problem is selected.

This technique guarantees a correct solution because it explores the **entire solution space** without skipping any possibility. However, exhaustive search is **computationally inefficient** since the number of possible solutions increases rapidly with input size, making it impractical for large problems.

Exhaustive search is mainly used when:

- The problem size is small
- Accuracy is more important than efficiency
- No better algorithm is available

Examples of Exhaustive Search

- **Travelling Salesman Problem:**
  All possible tours of cities are generated, and the tour with the minimum total travel cost is chosen.
- **Knapsack Problem:**
  All possible subsets of items are considered to find the subset that maximizes profit without exceeding the knapsack capacity.
- **Assignment Problem:**
  All possible assignments of jobs to machines are evaluated to determine the minimum-cost assignment.

25. Give the two major phases of performance evaluation.

The two major phases of performance evaluation are:

1. **A priori analysis** – theoretical analysis before implementation
2. **A posteriori analysis** – experimental analysis after implementation

26. What are the six steps in algorithmic problem solving?

The six steps are:

1. Understanding the problem
2. Deciding input and output
3. Selecting appropriate data structures
4. Designing the algorithm
5. Analyzing the algorithm
6. Implementing the algorithm

27. List the steps in mathematical analysis of non-recursive algorithms.

Steps include:

1. Identify the basic operation
2. Count the frequency of execution

3. Express the count using summation
4. Determine the order of growth

28. Explain linear search.

Linear search, also called **sequential search**, is a simple searching technique in which each element of the list is compared **one by one** with the given search key. The search continues until the element is found or until all elements in the list have been examined.

Linear search does **not require the data to be sorted**, which makes it easy to implement. However, it is **inefficient for large datasets** because, in the worst case, every element must be checked.

Algorithm: Linear Search

ALGORITHM LinearSearch(A, n, key)
Input: Array A of n elements, search key key
Output: Position of key if found, otherwise −1

```
for i ← 0 to n − 1 do
    if A[i] = key then
        return i
return −1
```

Analysis of Linear Search

- **Best Case:** O(1)
  (The key is found at the first position.)
- **Average Case:** O(n)
  (The key is found after checking about half the elements.)
- **Worst Case:** O(n)
  (The key is not present or found at the last position.)

29. Algorithm to find largest element in array and analyze.

Algorithm: Find Largest Element

ALGORITHM FindLargest(A, n)
Input: Array A of n elements
Output: Largest element in the array

max ← A[0]

for i ← 1 to n − 1 do

  if A[i] > max then

    max ← A[i]

return max

Complexity Analysis

- **Best Case:** O(n)
  Even in the best case, the algorithm must compare all elements to confirm the largest value.
- **Average Case:** O(n)
  On average, the algorithm performs n−1 comparisons to find the largest element.
- **Worst Case:** O(n)
  In the worst case, every element is compared once, resulting in linear time complexity.

If you want, I can do the **same format** for:

30. Algorithm for element uniqueness problem.

Algorithm: Element Uniqueness

ALGORITHM ElementUniqueness(A, n)
// Checks whether all elements in the array are distinct

Input: Array A of n elements
Output: TRUE if all elements are distinct, otherwise FALSE

for i ← 0 to n − 2 do

  for j ← i + 1 to n − 1 do

    if A[i] = A[j] then

      return FALSE

return TRUE

31. Algorithm for number of digits in binary representation.

Algorithm: BinaryDigitCount

ALGORITHM BinaryDigitCount(n)
Input: A positive decimal integer n
Output: Number of digits required to represent n in binary

count ← 0

while n > 0 do

  n ← n div 2

  count ← count + 1

return count

32. Steps in mathematical analysis of recursive algorithms.

The steps are:

1. Form the recurrence relation
2. Solve the recurrence relation
3. Determine the order of growth

33. Algorithm for factorial using recursion.

Step 1: Start

Step 2: Read n

Step 3: result ← FACT(n)

Step 4: Display result

Step 5: Stop

Function FACT(n)

  if n = 0 then

    return 1

  else

return n × FACT(n − 1)

34. Recursive algorithm for binary digits.

Step 1: Start

Step 2: Read n

Step 3: result ← COUNT(n)

Step 4: Display result

Step 5: Stop

Function COUNT(n)

if n = 0 then

return 0

else

return 1 + COUNT(n div 2)

35. Updated bubble sort and advantage.

Algorithm: Updated Bubble Sort

ALGORITHM UpdatedBubbleSort(A, n)
Input: Array A of n elements
Output: Sorted array in ascending order

Step 1: Start

Step 2: for i ← 0 to n − 2 do

swapped ← FALSE

for j ← 0 to n − 2 − i do

if A[j] > A[j + 1] then

swap A[j] and A[j + 1]

swapped ← TRUE

if swapped = FALSE then

break

Step 3: Display sorted array A

Step 4: Stop

Example

Consider the array: {1, 2, 3, 5, 4}

- In the first pass, only one swap occurs between 5 and 4.
- In the next pass, no swaps occur, so the algorithm stops early.

Advantage of Updated Bubble Sort over Normal Bubble Sort

The updated bubble sort **terminates early when no swaps occur**, thereby reducing unnecessary comparisons and improving efficiency for **already sorted or nearly sorted arrays**, unlike normal bubble sort which always performs all passes.

36. Prove the following theorem.

36. Prove the following theorem.

THEOREM: If $t_1(n) \in O(g_1(n))$ and $t_2(n) \in O(g_2(n))$, then $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$.
(The analogous assertions are true for the Ω and Θ notations as well.)

If
$t_1(n) \in O(g_1(n))$and
$t_2(n) \in O(g_2(n))$,

then

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

Proof

Since $t_1(n) \in O(g_1(n))$, there exist constants $c_1 > 0$and $n_1$such that

$$t_1(n) \leq c_1 g_1(n) \text{for all } n \geq n_1$$

Similarly, since $t_2(n) \in O(g_2(n))$, there exist constants $c_2 > 0$and $n_2$such that

$$t_2(n) \leq c_2 g_2(n) \text{for all } n \geq n_2$$

For all $n \geq \max(n_1, n_2)$,

$$t_1(n) + t_2(n) \leq c_1 g_1(n) + c_2 g_2(n)$$

Since

$$\max\{g_1(n), g_2(n)\} \geq g_1(n), g_2(n),$$

we get

$$t_1(n) + t_2(n) \leq (c_1 + c_2)\max\{g_1(n), g_2(n)\}$$

Hence,

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$$